

Authentication & Session 开发协作契约

Owner：A

依赖顺序：Wave 1；A 先合入公开入口和声明，Application Shell、Account Privacy、Account Center 随后并行消费。

定义完成：消费者可仅凭本文件的公开 seam 区分真实会话、确认状态和失败，并安全完成登录、退出和换号协作。

本文件是 Authentication & Session 唯一的跨 Owner 开发协作契约，也是人类阅读 PDF 的 Markdown 源。它固定公开 Dart 声明、结果语义、数据边界和联合验收；lib/features/authentication_session/ 内的 Widget、状态管理、Supabase SDK 映射、重试、私有文件和测试组织由 Owner 决定。

1. 任务成果、责任与依赖

- 用户可用邮箱密码登录当前设备；失败有可理解的修正或重试路径。
- 用户可注册；界面明确区分已建立会话、须查收验证邮件和未建立会话的失败；资料登记失败不否定认证结果。
- 账户页只依据 Auth 的真实邮箱与确认状态呈现；未知不显示“已验证”。
- 退出只结束当前设备会话；换号时先屏蔽和清理旧账户私有内容，不影响其他设备或远端业务记录。

Owner	负责	不负责	依赖
Authentication & Session (A)	邮箱密码认证、当前设备会话、真实邮箱/确认事实、可选注册资料、唯一公开入口	主应用门控/导航、私有 payload 清理、账户业务页面、其他设备会话	封装 Supabase Auth 和直接的 profiles 访问
Application Shell	认证门控、认证入口组合、退出/换号编排和私有 UI 屏障	Auth SDK 与认证规则	消费 AUTH-001；认证后调用 PRIVACY-001.open
Account Privacy	account scope 的 opened/closed 与本机清理	认证与导航	消费 AUTH-001 的 account id；仅 Shell 发起
Account Center	呈现真实邮箱/确认状态及退出意图	直读 Auth 或 profiles	消费 AUTH-001；退出交 Shell

包含登录、注册、真实邮箱确认、会话恢复/结束、可选注册资料及 ACCOUNT-07 的认证一侧。不包含 Magic Link、游客、忘记密码、固定 Verified User 文案和 Account Privacy 的清理实现。产品事实见[认证](#)、[账户](#)；系统流程见[FLOW-01](#)。

2. 需要调用的 Interface

Supabase Auth seam (AUTH-002，仅 A)

A 是唯一直接调用 Supabase Auth 的 Owner。SDK 方法、token 刷新、取消、重试与错误映射均为内部实现；其他 Owner 不可依赖。只有**明确有效或成功恢复**的会话可成为认证事实。过期缓存、刷新中、刷新失败、离线无法确认和远端拒绝都不得打开私有 scope。

直接访问对象	用途与不可变规则
auth.users	提供 account id、真实 email、email confirmed state 与当前设备会话。凭据/token 不复制到 public，且不由 profiles 推断邮箱或确认状态。
profiles	仅认证后写可选 username/avatar/bio；id 等于当前 Auth account id，owner-only CRUD。失败返回资料结果，不撤销或伪造认证结果。

字段、RLS 与迁移唯一权威为[Schema Catalog](#)。

3. 必须提供的 Interface

当前设备会话（AUTH-001）

提供者： Authentication & Session（A）

消费者： Application Shell、Account Privacy、Account Center

唯一公开 import： package:locatemy/features/authentication_session/authentication_session.dart

消费者只能 import 此入口，不得 import lib/features/authentication_session/src/。A 应先提交下列**声明**及最小公共类型；函数体由 A 亲自编写。

```

abstract interface class AuthenticationSession {
    Future<SessionSnapshot> restoreSession();
    Stream<SessionSnapshot> watchSession();
    Future<SignInOutcome> signIn({
        required String email, required String password,
    });
    Future<RegistrationOutcome> register({
        required String email,
        required String password,
        required String passwordConfirmation,
        String? username,
    });
    Future<SignOutOutcome> signOut();
}

sealed class SessionSnapshot {}
final class AuthenticatedSession extends SessionSnapshot {
    final AuthenticatedAccount account;
}
final class UnauthenticatedSession extends SessionSnapshot {}
final class SessionUnavailable extends SessionSnapshot {
    final SessionFailure failure;
}

final class AuthenticatedAccount {
    final String accountId;
    final String email;
    final EmailConfirmation confirmation;
}
enum EmailConfirmation { confirmed, verificationRequired, unavailable }

sealed class SignInOutcome {}
final class SignInSucceeded extends SignInOutcome {
    final AuthenticatedAccount account;
}
final class SignInRejected extends SignInOutcome {
    final SignInFailure failure;
}

sealed class RegistrationOutcome {}
final class RegistrationAuthenticated extends RegistrationOutcome {
    final AuthenticatedAccount account;
    final ProfileRegistrationOutcome profile;
}
final class RegistrationVerificationRequired extends RegistrationOutcome {
    final String email;
    final ProfileRegistrationOutcome profile;
}
final class RegistrationRejected extends RegistrationOutcome {
    final RegistrationFailure failure;
}

sealed class ProfileRegistrationOutcome {}
final class ProfileRegistered extends ProfileRegistrationOutcome {}
final class ProfileRegistrationSkipped extends ProfileRegistrationOutcome {}
final class ProfileRegistrationFailed extends ProfileRegistrationOutcome {
    final ProfileFailure failure;
}

sealed class SignOutOutcome {}
final class SignOutSucceeded extends SignOutOutcome {}
final class SignOutRejected extends SignOutOutcome {
    final SignOutFailure failure;
}

enum SessionFailure { retryableUnavailable, unsupportedClient, remoteRejected }
enum SignInFailure { invalidInput, invalidCredentials, retryableUnavailable, unsupportedClient }
enum RegistrationFailure { invalidInput, accountAlreadyExists, retryableUnavailable, unsupportedClient }
enum ProfileFailure { retryableUnavailable, permissionDenied, unknown }
enum SignOutFailure { retryableUnavailable, remoteRejected, unsupportedClient }

```

这规定协作形状，不规定 sealed-class 的内部文件组织、SDK enum 或错误文案。新增公共成员/结果变体须按第 7 节协商。

输入

调用	输入与约束	调用者责任
restoreSession / watchSession	无	Shell 只在认证门控未打开私有路由时调用；每个 snapshot 都重新决定是否可开 scope。
signIn	非空 email、password	密码仅本次表单/调用存在；不得记录、持久化或传给其他 Owner。
register	非空 email、password、passwordConfirmation；可空 username	confirmation 必须等于 password 才提交；无 username 表示跳过资料。密码强度与文案由 A 在产品事实范围内决定。
signOut	无	仅 Shell 在旧私有 UI 已屏蔽后调用；Account Center 不直接调用。

输出、失败与处理

结果	意义	消费者必须处理
AuthenticatedSession、SignInSucceeded、RegistrationAuthenticated	当前设备有明确有效会话；account 是不可变 Auth 身份和真实邮箱事实	Shell 以同一 accountId 调 PRIVACY-001.open，仅 opened 后建私有 UI；Account Center 使用 confirmation。
UnauthenticatedSession	明确没有可用会话	显示认证入口，不显示私有 UI。
SessionUnavailable	无法安全确认会话；failure 区分可重试网络、客户端不支持、远端拒绝	停在门控并给恢复路径；不得降级为未认证或开放私有 UI。
RegistrationVerificationRequired	注册须查收验证邮件；email 是注册目标邮箱	呈现查收邮件状态，不声称已验证；若后来有明确 session，仍以 SessionSnapshot 开 scope。
confirmed / verificationRequired / unavailable	Auth 确认、明确待验证、或无法给出确认事实	分别呈现已验证、待验证、状态不可得；未知不可猜测。
任一 *Rejected	本次动作没有对应成功语义	使用 enum 区分修正、重试或升级；不解析错误字符串，不建立 scope。
ProfileRegistrationFailed	认证/验证结果仍有效，只有资料未登记	显示资料可重试，绝不说成登录/注册失败。

watchSession 的首个和后续 snapshot 语义一致；过去 authenticated 值不能授权继续显示私有内容。

副作用、权限与调用次序

- accountId 唯一来自 Auth；公开类型不暴露密码、token、SDK payload 或 profiles 字段。
- 登录/注册/恢复只报告认证事实，不导航、不打开 scope、不清理别的模块数据。
- signOut 只结束当前设备会话；不会删除远端业务记录或结束其他设备会话。
- Shell 的退出/换号固定顺序：屏蔽旧私有 UI 和新业务意图 → signOut → PRIVACY-001.close(oldAccountId) → 两者成功后才显示普通登录入口。任一步失败仍无私有内容且可恢复。
- 已打开 scope 的 account id 与 AuthenticatedSession 不一致时，Shell 不为任何账户显示私有内容；按 FLOW-01 关闭旧范围后重新认证。

最小调用示例（Shell）

```
final snapshot = await authenticationSession.restoreSession();
switch (snapshot) {
  case AuthenticatedSession(:final account):
    // 以 account.accountId 调用 PRIVACY-001.open；仅 opened 后进入主应用。
  case UnauthenticatedSession():
    // 显示登录/注册入口。
  case SessionUnavailable(:final failure):
    // 保持门控，按 failure 提供恢复路径。
}
```

这是 seam 用法说明，不是可提交的 Shell 实现。

Fake Adapter 场景

Shell 用 fake AuthenticationSession 分别返回 AuthenticatedSession(account A)、UnauthenticatedSession、SessionUnavailable(retryableUnavailable)，验证仅在 A 的 PRIVACY-001.open 返回 opened 后建立私有 UI。Account Center 用同一 fake 的三种 EmailConfirmation，验证 unavailable 不会被呈现为 verified。二者无需 Supabase 或 A 的生产实现。

4. 推荐实现顺序

- 1. **A：先合入公开 seam。** 创建唯一入口、上述声明及最小 fake；合入前消费者不依赖 A 内部文件。
- 2. **A：实现真实 Adapter 与表单。** 映射 Supabase Auth/profiles 到这些结果，并由 A 编写真实 Adapter 契约测试。
- 3. **消费者：并行使用 fake。** Shell 完成门控和 FLOW-01；Account Privacy 只接收 account id；Account Center 只用 AuthenticatedAccount。
- 4. **共同联调。** 接入生产 seam，仅为第 5 节保留必要跨模块流测试。

5. 联合验收

场景	参与 Owner	操作	可观察结果	追踪
冷启动恢复会话	A、Shell、Privacy	无会话、有效 A、不可确认会话	分别为认证入口、仅 scope opened 后主应用、无私有内容的恢复门控	AUTH-01；AT-AUTH-01
登录凭据或网络失败	A、Shell	非法/错误凭据、网络不可用、客户端不支持	可区分失败和恢复路径；无 scope；密码不泄露	AUTH-01；AT-AUTH-01、AT-OUT-02
注册与真实确认	A、Shell、Account Center	authenticated、verification required、资料失败	认证/验证/资料状态分明；仅 Auth 确认事实可显示 verified	AUTH-02、AUTH-03；AT-AUTH-02
当前设备退出	A、Shell、Privacy	确认退出；注入 Auth/清理失败	成功到普通登录；失败仍无私有 UI、可重试；远端/其他设备不变	ACCOUNT-07；AT-OUT-01、AT-OUT-02
从 A 换到 B	A、Shell、Privacy、私有 Owner	A 已打开后退出并登录 B	B scope opened 前 A 数据不可读；随后只呈现 B 数据	ACCOUNT-07；AT-SWITCH-01

6. 实现自由、阻塞项与参考

Owner 可决定 src/ 内的拆分、Widget、状态管理、SDK Adapter、刷新/重试/取消、局部校验和测试组织。以下情况必须暂停协商：变更唯一公开入口、公共声明/结果/次序/权限；需变更 profiles schema/RLS；或 Supabase 版本使 RISK-SESSION-01 的锁定结论失效。

当前阻塞项：无。依赖版本变化时重开 RISK-SESSION-01。权威参考：[Schema Catalog](#)、[Interface 注册表](#)、[会话风险](#)。

7. 契约变更与完成检查

公共 Interface 变更须由提供方说明原因和受影响消费者，所有受影响消费者确认；同一 PR 更新公开声明、本契约、受影响 fake/Adapter 测试和 PDF。Git/PR 保存历史；不使用文档版本、checksum、Manifest、Generation Gate 或 Development Release。

- [x] Owner、消费者、依赖顺序和唯一公开入口明确。
- [x] AUTH-001 的声明、输入、结果、失败、副作用、权限、次序、示例和 fake 场景完整。
- [x] 直接数据对象及其权限有唯一来源；消费者不接触 persistence schema。
- [x] 联合验收覆盖恢复/门控、注册/验证、退出/清理和 A→B 换号。